

Projekt SK Okres 3-4

Krzysztof Bierzanek, Igor Starnowski

Wykrywanie opóźnień

```
    }  
    case PAKIET_PING: {  
        QByteArray dane;  
        QDataStream out(&dane, QIODevice::WriteOnly);  
        out.setVersion(QDataStream::Qt_6_0);  
        out << (quint32)0;  
        out << (quint8)PAKIET_PONG;  
        out << czasWyslania;  
        out.device()->seek(0);  
        out << (quint32)dane.size();  
        m_socket.write(dane);  
        m_socket.flush();  
        break;  
    }  
    case PAKIET_PONG: {  
        qint64 obecnyCzas = QDateTime::currentMSecsSinceEpoch();  
        qint64 rtt = obecnyCzas - czasWyslania;  
        emit nowyPing(rtt / 2);  
        break;  
    }  
    default:
```

Radzenie sobie z opóźnieniami

```
void MainWindow::odbierzPing(qint64 ping) {
    ui->labelPing->setText(QString("Ping: %1 ms").arg(ping));
    int interval = ui->spinInterval->value();
    if (ping > interval) {
        ui->labelPing->setStyleSheet("color: white; background-color: red; font-weight: bold; padding: 2px;");
        if (!ui->btnStart->isEnabled()) {
            if (klient != nullptr) klient->rozlacz();
            if (serwer != nullptr) serwer->zatrzymaj();
            QMessageBox::warning(this, "Awaryjne Przejscie w Tryb Lokalny",
                                "Opóźnienie sieci (" + QString::number(ping) + " ms) "
                                "jest większe niż interwał symulacji (" + QString::number(interval) + " ms)!\n\n"
                                "Połączenie sieciowe zostało zerwane, aby zapobiec desynchronizacji. "
                                "Symulacja NIE została zatrzymana i jest kontynuowana na "
                                "ostatnich odebranych parametrach (Tryb Lokalny).");
        }
    } else if (ping > interval * 0.7) {
        ui->labelPing->setStyleSheet("color: black; background-color: yellow; font-weight: bold; padding: 2px;");
    } else {
        ui->labelPing->setStyleSheet("color: white; background-color: green; font-weight: bold; padding: 2px;");
    }
}
```

Końcowy projekt protokołu sieciowego

Wartość	ID	Identyfikator	Zawartość	Funkcja
0x01	1	KONF_PID	RegulatorPID	Inicjalizacja/zmiana nastaw regulatora
0x02	2	KONF_ARX	ModelARX	Przekazanie parametrów i limitów obiektu
0x03	3	KONF_GEN	Generator	Parametry wartości zadanej
0x0A	10	KONF_INT	Interwał	Przekazanie interwału
0x10	16	PROBKI_SYGNAL	Próbki sygnałów	Bieżące wartości: t, w, e, u, y, wysyłane na wykresy
0x20	32	SYM_KONTROLKI	Komendy sterujące	Kontrola silnika symulacji
0x63	99	PAKIET_PING	timestamp	Sprawdzenie połączenia i pomiar opóźnienia
0x64	100	PAKIET_PONG	timestamp	Odpowiedź na pakiet PING

Sposób serializacji

```
void klientTCP::onReadyRead()  
{  
    QDataStream in(&m_socket);  
    in.setVersion(QDataStream::Qt_6_0);  
  
    while (true) {
```

```
        case KONF_ARX: {  
            ModelARX odebranyARX;  
            in >> odebranyARX;  
            emit otrzymanoNowyARX(odebranyARX);  
            break;  
        }
```